

Course MA-IMF 1225 - "Lab Exploring HPC technologies"

Analysis of AMPM and IMP Hardware Prefetchers on an Out-of-Order Armv8 Architecture

Faruk Demirel
Jon Lumi
Computer Science Department
University of Bonn



July 05, 2026

Contents

List of Tables

List of Figures

Abstract

Hardware prefetching is an important technique for reducing the performance impact of long memory access latencies in modern processors. This project evaluates several hardware data prefetchers for scientific and memory-intensive benchmark kernels on a simulated out-of-order Armv8 architecture using gem5. The main focus is on the Access Map Pattern Matching (AMPM) prefetcher and the Indirect Memory Prefetcher (IMP), which are compared against simpler Next-line and Stride prefetchers. The evaluation considers prefetchers placed at both the L1 data cache and the shared L2 cache, using two DDR4 memory configurations. Several benchmark kernels are studied, including STREAM Triad, sparse matrix-vector multiplication, matrix multiplication, merge sort, quick sort, and breadth-first search.

The report describes the simulated architecture, benchmark selection, prefetcher configurations, and evaluation metrics. The experimental analysis focuses on speedup, prefetch accuracy, coverage, timely and late prefetches, unused prefetches, and cache miss behavior. The goal is to understand when more complex prefetchers such as AMPM and IMP provide benefits over simpler spatial prefetchers, and how prefetcher behavior changes depending on benchmark access patterns, cache placement, and memory bandwidth.

1 Introduction

Memory performance is a major bottleneck in many high-performance computing (HPC) applications. While modern processors continue to increase core counts, instruction throughput, and vector capabilities, memory latency and bandwidth often improve more slowly. As a result, applications with large working sets or irregular memory accesses may spend a significant fraction of execution time waiting for data to arrive from the memory hierarchy.

Hardware prefetching is one technique used to reduce this bottleneck. A prefetcher tries to predict future memory accesses and fetch the corresponding cache lines before they are demanded by the processor. If the prediction is correct and timely, the demand access can hit in the cache instead of stalling on a memory access. However, prefetching can also be harmful if it fetches useless cache lines, consumes memory bandwidth, or evicts useful data from the cache.

This project studies hardware prefetching for a set of scientific and memory-intensive benchmarks using the gem5 simulator. The focus is on two more advanced prefetching techniques: Access Map Pattern Matching (AMPM) and the Indirect Memory Prefetcher (IMP). AMPM is designed to identify spatial access patterns within memory regions, while IMP targets indirect memory accesses such as indexed references. These prefetchers are compared with two simpler baseline prefetchers: Next-line and Stride.

The evaluation is performed on an out-of-order Armv8 architecture model. Prefetchers are evaluated at both the L1 data cache and L2 cache, and experiments are repeated for two DDR4 memory configurations. The main goal is to analyze how different prefetchers behave across benchmarks with different memory access patterns, and to determine when complex prefetchers provide an advantage over simpler spatial prefetching mechanisms.

The rest of this report is organized as follows. Section ?? introduces background on memory prefetching and the evaluated prefetchers. Section ?? describes the simulation methodology, architecture model, benchmarks, prefetcher configurations, and metrics. Section ?? presents the evaluation results, including speedup, cache miss behavior, prefetch accuracy, coverage, and timeliness. Finally, Section ?? summarizes the main findings and lessons learned. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2 Background & Related Work

Hardware data prefetching attempts to hide memory latency by predicting future data accesses and issuing memory requests before the corresponding load instructions require the data. A successful prefetch should be both accurate and timely: it should fetch a cache line that will actually be used, and it should arrive early enough to reduce or eliminate the demand miss latency.

Different prefetchers target different types of memory access patterns. Simple spatial prefetchers, such as Next-line, assume that future accesses are close to the current access. A Next-line prefetcher fetches the following cache line after an access, without requiring complex pattern detection. This can be effective for streaming or sequential access patterns, such as array traversal.

Stride prefetchers detect constant-distance access patterns. If a sequence of accesses follows a pattern such as consecutive cache lines or a fixed stride, the prefetcher can predict future addresses by adding the detected stride to the current address. Stride prefetching is especially relevant for scientific codes with regular array-based memory access patterns.

Access Map Pattern Matching (AMPM) is a more advanced spatial prefetcher. Instead of only tracking one stride per instruction, AMPM records access patterns within memory regions and attempts to identify multiple spatial relationships. This makes it suitable for workloads with interleaved or more complex spatial patterns. In this project, AMPM is evaluated as a representative spatial prefetcher with higher complexity than Next-line and Stride.

The Indirect Memory Prefetcher (IMP) targets indirect memory access patterns. These patterns occur when one memory access provides an index or pointer used by a later memory access. A common example is an access of the form $A[B[i]]$. Such patterns appear in sparse matrix computations, graph algorithms, and other irregular workloads. Unlike simple spatial prefetchers, IMP is intended to capture dependence-based memory behavior rather than only fixed address distances.

The evaluated benchmark set includes both regular and irregular memory access behavior. STREAM Triad and matrix multiplication contain more regular array-based accesses. Sparse matrix-vector multiplication and breadth-first search include indirect or irregular components, although their concrete implementations may still expose spatial locality through compressed or contiguous data structures. Merge sort and quick sort provide additional memory-intensive workloads with different access and control-flow behavior.

Prior work on hardware prefetching has shown that no single prefetcher is optimal for all access patterns. Aggressive prefetchers may improve performance by increasing memory-level parallelism, but they can also reduce accuracy and increase memory traffic. Therefore, this project evaluates not only speedup, but also prefetch accuracy, coverage, timeliness, unused prefetches, and cache miss behavior. **Example of citing a related work:** Santhosh et al. described a technique to dynamically adapt prefetch aggressiveness at runtime [?].

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non

justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

3 Evaluation Methodology

This section describes the simulation setup used to evaluate the hardware prefetchers. The evaluation uses gem5 to simulate an out-of-order Armv8 system. The experiments compare AMPM and IMP against Next-line and Stride prefetchers across several benchmark kernels, cache placements, and memory configurations.

3.1 Architecture Model

The simulated system uses an out-of-order Armv8 CPU model. The core is configured with a 4-wide pipeline, a reorder buffer, physical integer and floating-point/vector registers, and a load-store queue. This setup allows memory operations to be reordered dynamically, which is important for realistic prefetcher evaluation because out-of-order execution can interleave memory streams and make access patterns more difficult to detect.

The cache hierarchy consists of private L1 instruction and data caches and a shared L2 cache. The L1 data cache and L2 cache are both evaluated as possible prefetcher insertion points. For each prefetcher family, experiments are performed with the prefetcher attached either to the L1 data cache or to the L2 cache, while the other cache level does not use that prefetcher.

Two DDR4 memory configurations are evaluated. The first configuration represents the baseline DDR4 memory setup, while the second configuration increases available memory bandwidth. These two memory configurations are used to study how prefetcher effectiveness changes when more memory bandwidth is available. This is important because aggressive prefetching can generate additional memory traffic, and its benefit may depend on whether the memory system can absorb this traffic.

3.2 Evaluated Prefetchers

Four prefetcher families are evaluated: Next-line, Stride, AMPM, and IMP. Next-line and Stride are used as simpler reference prefetchers. AMPM and IMP are the main focus of the project.

The Next-line prefetcher represents a simple spatial prefetching strategy. It assumes that future memory accesses are likely to continue to the next cache line. This prefetcher has low complexity and can be effective for streaming memory accesses.

Table 1: Simulated architecture parameters.

Component	Configuration
ISA	Armv8
CPU model	Out-of-order core
Pipeline width	Placeholder
ROB size	Placeholder
Integer registers	Placeholder
FP/vector registers	Placeholder
Load-store queue size	Placeholder
L1 data cache	Placeholder
L2 cache	Placeholder
Memory configurations	DDR4 1x and DDR4 2x
Simulator	gem5

The Stride prefetcher detects regular address differences between memory accesses. Once a stable stride is detected, it issues prefetch requests for future addresses following the same stride. This prefetcher is expected to perform well for regular array-based access patterns.

AMPM tracks accesses within memory regions and attempts to identify spatial patterns from the observed access map. Compared with Next-line and Stride, AMPM can potentially detect more complex spatial patterns and multiple interleaved streams.

IMP targets indirect memory access patterns, where one loaded value is used to compute the address of a later memory access. This makes IMP relevant for sparse and graph-based workloads, where indexed memory references are common.

For each prefetcher, both default and tuned configurations are considered where applicable. The tuned configurations vary relevant prefetcher parameters such as prefetch degree, distance, thresholds, table sizes, and other prefetcher-specific parameters. The final comparison reports both default behavior and the best-performing tuned configuration.

Table 2: Evaluated prefetcher families.

Prefetcher	Main target pattern	Configuration exploration
Next-line	Sequential/spatial accesses	Default and tuned parameters
Stride	Constant-stride accesses	Default and tuned parameters
AMPM	Spatial access-map patterns	Placeholder for explored parameters
IMP	Indirect memory accesses	Placeholder for explored parameters

3.3 Benchmarks

The evaluation uses six benchmark kernels with different memory access characteristics. The benchmark set includes regular streaming behavior, dense computation, sparse computation, sorting, and graph traversal. This allows the prefetchers to be evaluated across a range of access patterns.

Table 3: Benchmark selection.

Benchmark	Description	Expected memory behavior
STREAM Triad	Vector streaming benchmark	Regular sequential accesses
SpMV	Sparse matrix-vector multiplication	Sparse and indirect accesses
Matrix multiplication	Dense matrix computation	Regular array accesses
Merge sort	Sorting benchmark	Data-dependent memory behavior
Quick sort	Sorting benchmark	Data-dependent memory behavior
BFS	Breadth-first search graph traversal	Graph/indirect accesses with adjacency

Although BFS is algorithmically irregular, the implementation used in this project stores graph data in a compact adjacency-array format. Therefore, part of its execution scans the edge array sequentially. This means that BFS can expose both irregular accesses and spatially predictable memory streams. This is relevant when interpreting the performance of IMP compared with Next-line and Stride.

3.4 Experimental Design

Each benchmark is simulated under multiple prefetcher configurations. The experiments vary the prefetcher family, prefetcher configuration, cache placement, and memory configuration. A no-prefetch baseline is also simulated for each benchmark and memory configuration.

The primary comparison is speedup relative to the no-prefetch baseline. For each benchmark and system configuration, the best-performing configuration of each prefetcher family is selected. In addition, the default configuration of each prefetcher family is recorded and shown as a reference. This allows the analysis to distinguish between the default behavior of a prefetcher and the improvement obtained through parameter tuning.

The full experimental matrix is summarized as follows:

- Benchmarks: STREAM Triad, SpMV, matrix multiplication, merge sort, quick sort, BFS.
- Prefetcher families: Next-line, Stride, AMPM, IMP.
- Cache placement: L1 data cache and L2 cache.
- Memory configurations: DDR4 1x and DDR4 2x.
- Baseline: no hardware data prefetching.
- Evaluation modes: default prefetcher configuration and tuned configurations.

3.5 Metrics

Several metrics are used to evaluate prefetcher behavior. Speedup measures the overall performance improvement relative to the no-prefetch baseline. Cache miss rates are used

to determine whether a prefetcher reduces demand misses. Prefetch accuracy and coverage are used to evaluate how effectively the prefetcher predicts useful memory accesses. Timely, late, and unused prefetches are used to distinguish between useful prefetches that arrive early enough, useful prefetches that arrive too late, and prefetches that are never used.

Table 4: Evaluation metrics.

Metric	Description
Simulation time	Runtime reported by gem5 for the benchmark region
Speedup	No-prefetch simulation time divided by prefetcher simulation time
IPC	Instructions per cycle, used to observe core-level performance changes
Demand misses	Cache misses caused by demand accesses
Miss rate	Demand misses divided by demand accesses
Prefetches issued	Number of prefetch requests generated by the prefetcher
Useful prefetches	Prefetched cache lines that are later used by demand accesses
Unused prefetches	Prefetched cache lines that are not used before eviction
Prefetch accuracy	Fraction of issued prefetches that are useful
Prefetch coverage	Fraction of potential demand misses covered by useful prefetches
Timely prefetches	Useful prefetches that arrive before the demand access
Late prefetches	Useful prefetches that arrive too late to fully hide latency

The main speedup metric is calculated relative to the no-prefetch baseline:

$$\text{Speedup} = \frac{T_{\text{no-prefetch}}}{T_{\text{prefetcher}}} \quad (1)$$

where $T_{\text{no-prefetch}}$ is the simulation time without hardware prefetching and $T_{\text{prefetcher}}$ is the simulation time with a given prefetcher configuration.

Prefetch accuracy is calculated as:

$$\text{Accuracy} = \frac{\text{Useful prefetches}}{\text{Issued prefetches}} \quad (2)$$

Prefetch coverage is calculated as:

$$\text{Coverage} = \frac{\text{Useful prefetches}}{\text{Useful prefetches} + \text{Demand misses}} \quad (3)$$

These metrics are analyzed together because high speedup does not necessarily imply high accuracy. A prefetcher may improve performance despite lower accuracy if it generates enough timely prefetches and the memory system has sufficient bandwidth. Conversely, a prefetcher may have high accuracy but low speedup if its prefetches arrive too late or if it does not issue enough requests.

3.6 Architecture model

Describe the simulated CPU architecture. Describe also a range of the parameters of the specific hardware components (e.g., prefetchers, branch predictor, SVE vector length) if they are used in your evaluation and your exploration.

Example of referring to a figure: Figure ?? is an example



Figure 1: An example.

3.7 Benchmarks

The evaluation used the benchmark listed in Table ?? . For the project focus on the DLP topic, the description of SVE implementation for each specific code should be described in this section too.

Table 5: Benchmarks selection.

	Description
Stream-Triad	Stream benchmark

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

3.8 Metrics

Describe shortly the metrics used in your evaluation, .e.g. as in Table ?? .

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie

Table 6: Metrics.

	Description
Speedup	
IPC	
prefetch accuracy	

vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

4 Evaluation Results

This section presents the experimental results. The analysis is organized in several parts. First, the overall speedup of each prefetcher family is compared against the no-prefetch baseline. Second, default configurations are compared with tuned configurations. Third, cache miss behavior is analyzed. Finally, prefetch accuracy, coverage, timeliness, and unused prefetches are examined to explain the observed speedup trends.

4.1 Overall Speedup Comparison

Figure ?? will show the best-performing configuration of each prefetcher family for each benchmark, cache placement, and memory configuration. The no-prefetch baseline is normalized to a speedup of 1.0.

The expected analysis in this subsection is to compare Next-line, Stride, AMPM, and IMP across all benchmarks. Particular attention should be given to whether AMPM and IMP outperform simpler prefetchers, and whether the winning prefetcher depends on the benchmark memory access pattern.

For regular streaming benchmarks such as STREAM Triad, simple spatial prefetchers are expected to perform strongly because the access pattern is easy to predict. For sparse and indirect workloads such as SpMV, IMP may have more opportunity to improve performance. BFS requires a more careful interpretation because the implementation contains both irregular graph traversal behavior and sequential scans through the adjacency array.

4.2 Default Versus Tuned Configurations

In addition to comparing the best tuned configurations, the default configuration of each prefetcher is used as a reference. This allows the evaluation to show whether performance

improvements come from the basic prefetching technique itself or from parameter tuning.

The plots in this subsection will show the best tuned configuration as a bar and the default configuration as a marker. This is useful for identifying benchmarks where tuning provides a large benefit and benchmarks where the default configuration is already close to optimal.

The discussion should identify cases where the default configuration performs close to the best configuration, as well as cases where tuning significantly improves speedup. This is especially important for IMP and AMPM, where parameter choices can strongly affect aggressiveness, coverage, and usefulness.

4.3 Effect of Cache Placement

This subsection compares prefetching at the L1 data cache with prefetching at the L2 cache. Prefetching at L1D can potentially reduce latency earlier in the hierarchy, but it may also increase pressure on the small L1 cache. Prefetching at L2 has more cache capacity available and may avoid some L1 pollution, but it may be less effective if prefetches arrive too late for the core.

The analysis should compare the same benchmark and prefetcher family across L1D and L2 placement. Important questions include:

- Does a prefetcher perform better at L1D or L2?
- Does the preferred placement depend on benchmark access pattern?
- Does L2 placement reduce harmful cache pollution?
- Does L1D placement improve timeliness?

4.4 Effect of Memory Bandwidth

This subsection compares DDR4 1x and DDR4 2x memory configurations. Prefetching can increase memory-level parallelism and hide latency, but aggressive prefetching can also generate additional traffic. Therefore, the effectiveness of a prefetcher may depend on available memory bandwidth.

The analysis should identify whether increasing memory bandwidth improves the benefit of aggressive prefetching. This is particularly relevant for memory-intensive benchmarks such as STREAM Triad and SpMV. If a benchmark is bandwidth-limited, higher bandwidth may allow more prefetch requests to be useful. If the memory system is already saturated, additional prefetch traffic may provide limited benefit.

4.5 Cache Miss Rate Analysis

Speedup should be interpreted together with cache miss behavior. A prefetcher is expected to improve performance by reducing demand misses, especially at the cache level where it is installed. However, a reduction in miss rate does not always translate directly into speedup, because prefetch timeliness and memory bandwidth also matter.

This subsection will analyze L1D and L2 demand miss rates for each prefetcher family. The analysis should compare miss rates against the no-prefetch baseline and explain whether the speedup trends match the miss-rate reductions.

Important points to discuss include:

- Which prefetchers reduce demand misses the most?
- Are there cases where miss rate improves but speedup remains small?
- Are there cases where prefetching increases cache traffic or cache pollution?
- Does L1D or L2 placement produce stronger miss-rate reduction?

4.6 Prefetch Accuracy and Coverage

Prefetch accuracy and coverage explain how well a prefetcher predicts useful cache lines. Accuracy measures the fraction of issued prefetches that are later used. Coverage measures how much of the demand-miss stream is covered by useful prefetches.

A high-accuracy prefetcher is conservative and avoids useless prefetches, but may not issue enough requests to cover many misses. A lower-accuracy prefetcher may still improve performance if it provides high coverage and enough timely prefetches. Therefore, accuracy and coverage must be analyzed together.

This subsection should compare Next-line, Stride, AMPM, and IMP in terms of accuracy and coverage. For example, Next-line may have lower accuracy for irregular workloads but still produce good speedup if many of its prefetches are timely and the memory system has sufficient bandwidth. IMP may show higher usefulness for indirect memory patterns, especially when the benchmark exposes enough dependent address behavior.

4.7 Timely, Late, and Unused Prefetches

A useful prefetch improves performance only if it arrives early enough. Therefore, prefetches are separated into timely, late, and unused categories. Timely prefetches arrive before the demand access and can hide latency. Late prefetches are useful but arrive too late to fully hide latency. Unused prefetches consume bandwidth and cache capacity without helping performance.

This subsection should explain whether the speedup results are mainly caused by an increase in timely prefetches, a reduction in demand misses, or a change in unused prefetches. It should also identify cases where a prefetcher issues many requests but does not improve performance because many prefetches are late or unused.

4.8 Per-Benchmark Discussion

This subsection summarizes the behavior of each benchmark individually.

4.8.1 STREAM Triad

STREAM Triad is expected to favor simple spatial prefetchers because it contains regular streaming memory accesses. The analysis should compare whether Next-line and Stride outperform AMPM and IMP, and whether higher memory bandwidth improves the benefit of aggressive prefetching.

4.8.2 SpMV

SpMV contains sparse and indirect memory behavior, making it an important benchmark for IMP. The analysis should determine whether IMP provides an advantage over spatial prefetchers, and whether AMPM can exploit spatial structure in the sparse matrix representation. Coverage and timeliness are especially important for this benchmark.

4.8.3 Matrix Multiplication

Matrix multiplication has regular array-based behavior but may also have cache reuse depending on the implementation and problem size. The analysis should explain whether prefetchers provide only modest improvement or whether specific configurations reduce cache misses significantly.

4.8.4 Merge Sort

Merge sort has data movement and comparison behavior that may expose sequential accesses during merging. The analysis should determine whether spatial prefetchers are sufficient or whether AMPM/IMP provide additional benefit.

4.8.5 Quick Sort

Quick sort has data-dependent partitioning behavior. The analysis should examine whether prefetchers improve performance consistently or whether irregular control flow limits prefetch effectiveness.

4.8.6 BFS

BFS is algorithmically irregular, but the implementation uses an adjacency-array representation. Therefore, the edge traversal contains sequential scans through the edge array, while accesses to the visited array are indirect. The analysis should explain whether the observed performance is dominated by the sequential edge traversal or by the indirect visited-array accesses. This is important for understanding why IMP may or may not outperform Next-line and Stride. Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

5 Conclusion

This project evaluated hardware data prefetching for a set of scientific and memory-intensive benchmarks on a simulated out-of-order Armv8 architecture. The main focus was on AMPM and IMP, which were compared against simpler Next-line and Stride prefetchers. The evaluation considered both L1D and L2 prefetcher placement, two DDR4 memory configurations, and both default and tuned prefetcher configurations.

The results show that prefetcher effectiveness depends strongly on benchmark memory access patterns. Simple spatial prefetchers can be highly effective for workloads with regular or streaming behavior. More complex prefetchers such as AMPM and IMP are expected to provide benefits when the benchmark exposes the type of access pattern they are designed for, such as multiple spatial streams or indirect memory references.

A key lesson is that algorithmic irregularity does not always imply that an indirect prefetcher will dominate. For example, a BFS implementation using contiguous adjacency arrays may still expose strong spatial locality during edge traversal, allowing Next-line or Stride prefetchers to perform well. Similarly, sparse workloads may contain both indirect accesses and regular streams, so the best prefetcher depends on which part of the memory behavior dominates execution time.

The evaluation also highlights the importance of analyzing more than speedup alone. Cache miss rates, prefetch accuracy, coverage, timely prefetches, late prefetches, and unused prefetches are necessary to explain why a prefetcher improves or fails to improve performance. A prefetcher with lower accuracy may still perform well if it produces many timely prefetches and the memory system has sufficient bandwidth. Conversely, a prefetcher with high accuracy may provide limited speedup if its prefetches arrive too late or cover too few misses.

The final report will complete this analysis by adding the AMPM results, the final Stage 3 IMP results, and detailed plots for speedup, default-versus-tuned behavior, cache miss rates, prefetch coverage, accuracy, timeliness, and unused prefetches. Quisque ullamcorper

placemat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.



images/placeholder.png

Figure 2: Placeholder: best prefetcher-family comparison against the no-prefetch baseline.

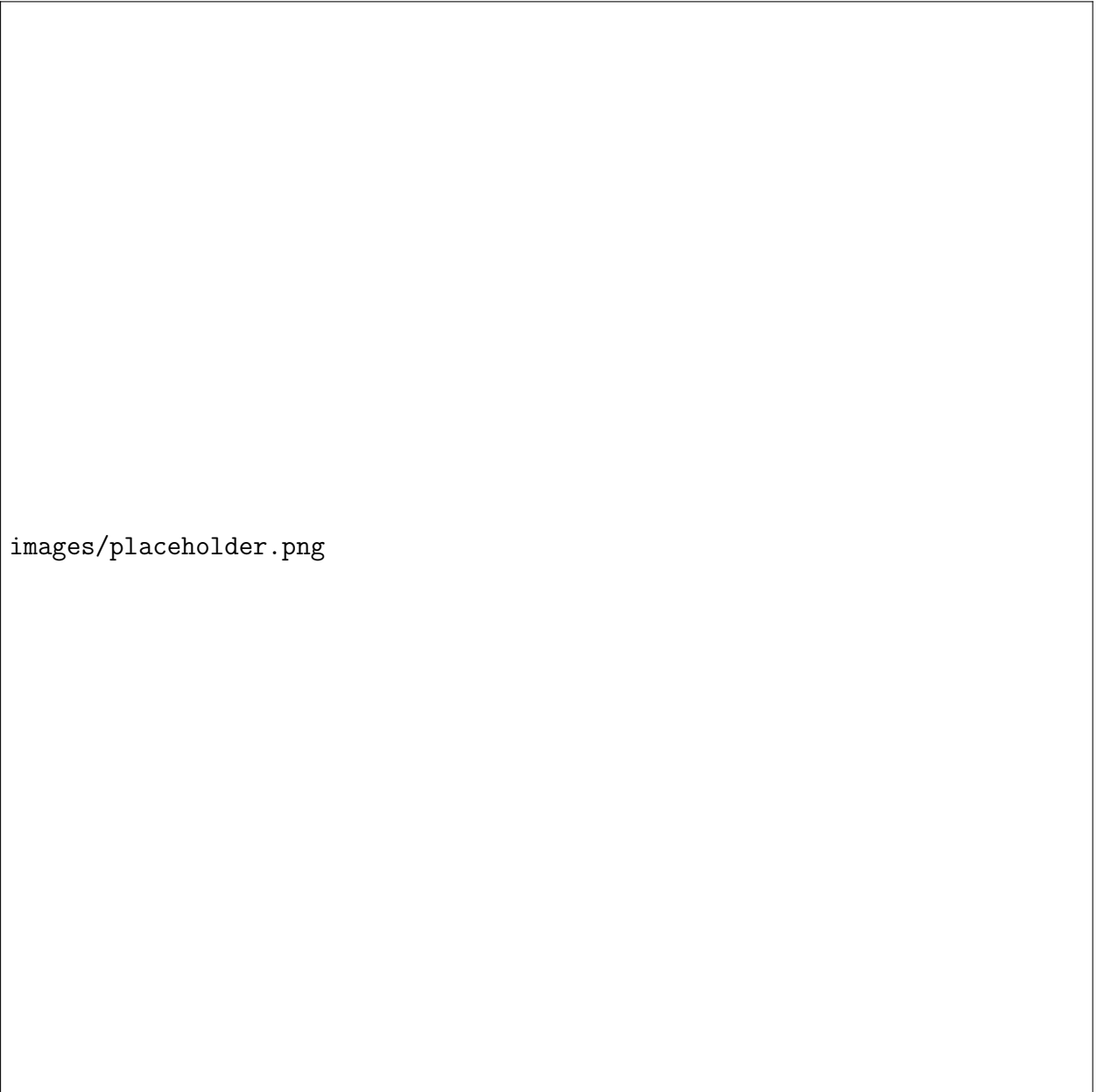


Figure 3: Placeholder: tuned prefetcher performance with default configuration markers.

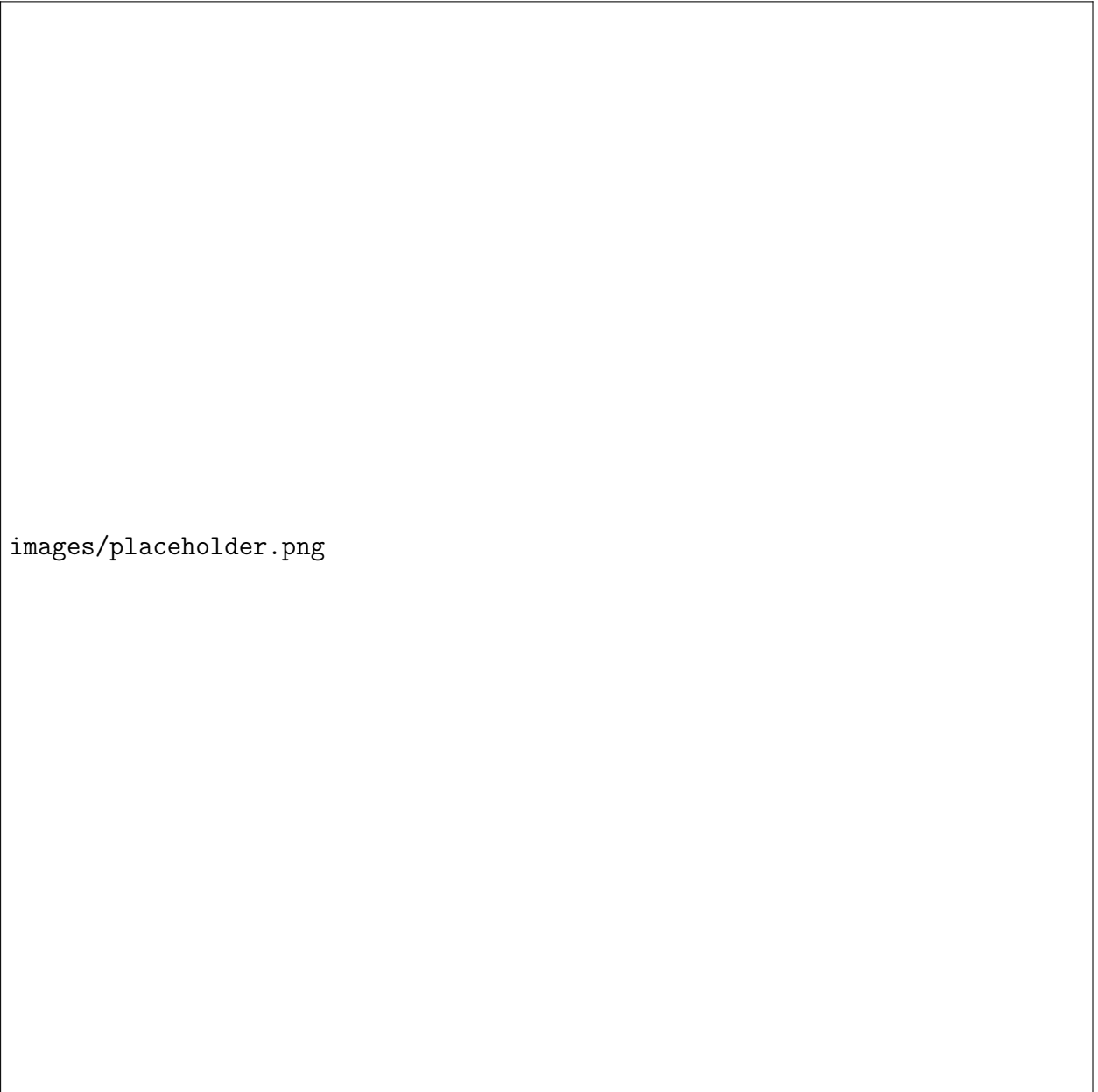


Figure 4: Placeholder: L1D and L2 demand miss-rate comparison.

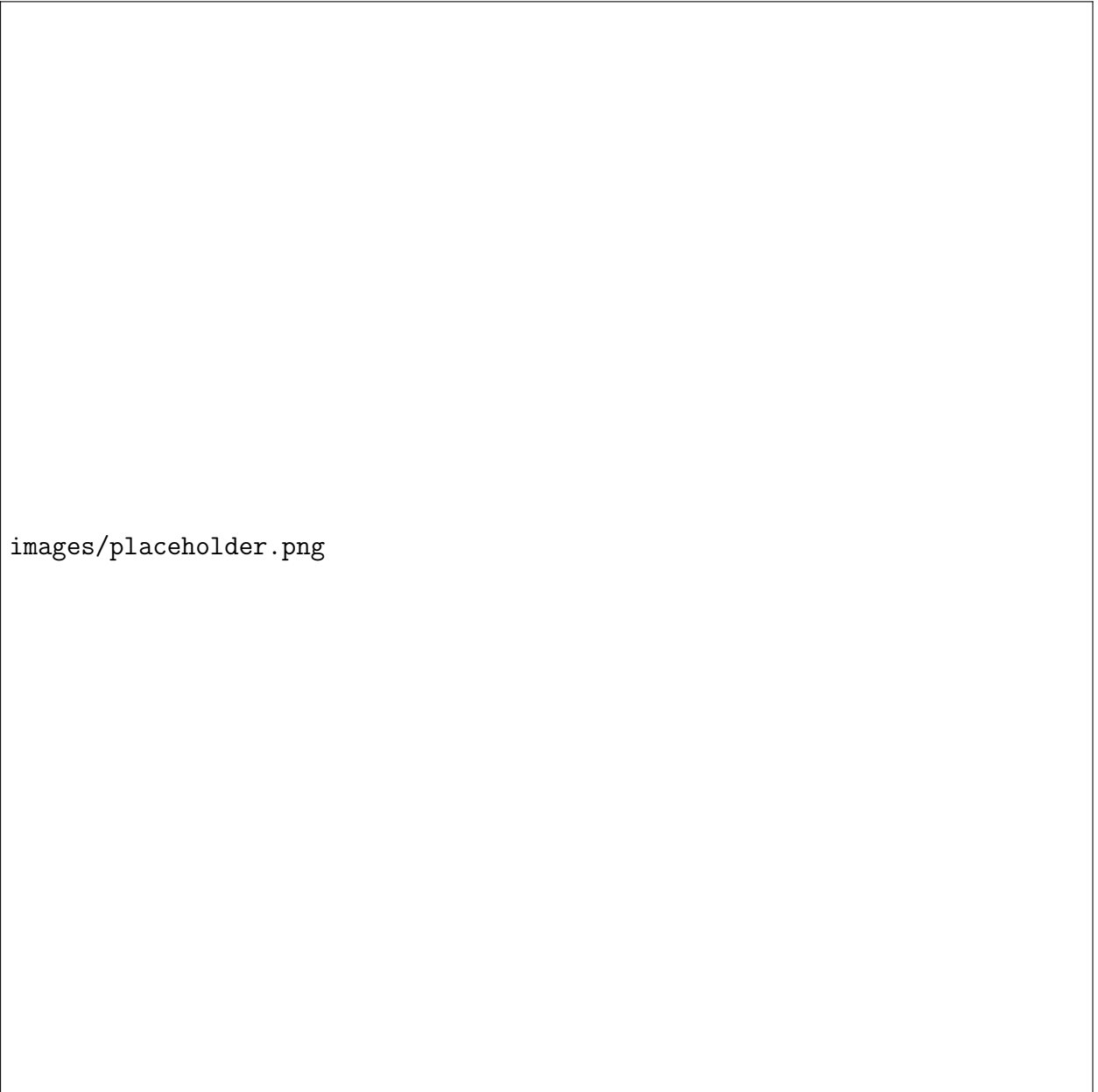


Figure 5: Placeholder: prefetch accuracy and coverage comparison.

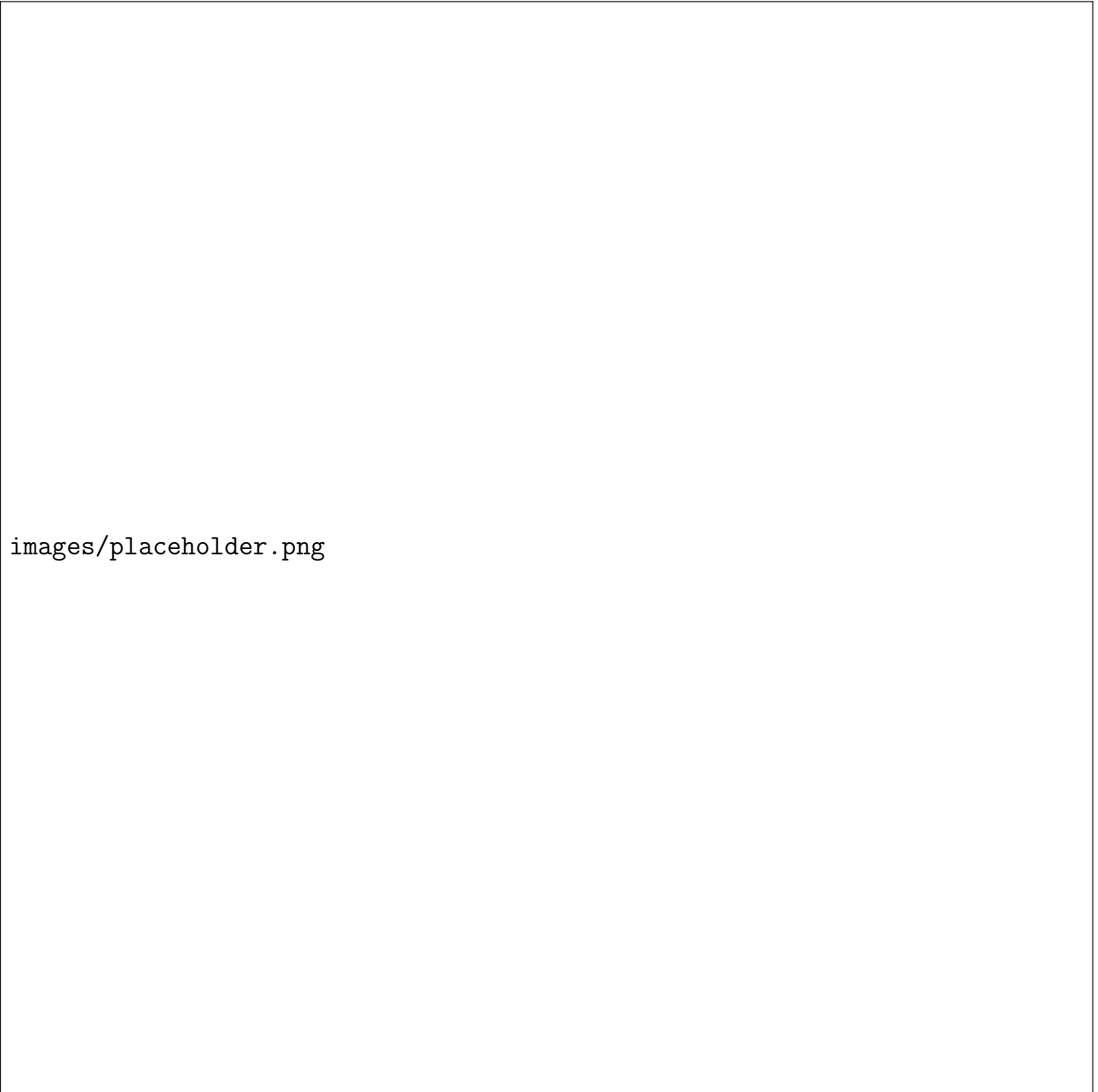


Figure 6: Placeholder: timely, late, and unused prefetches.